



```
In [1]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
In [2]: import torch
import torch.nn as nn
import torch.optim as optim
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
```

```
In [9]: dataset1 = pd.read_csv("/content/drive/MyDrive/exp-1 - Sheet1.csv")
X = dataset1[['input']].values
y = dataset1[['output']].values
```

```
In [10]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42)
```

```
In [11]: scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
In [12]: X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32).view(-1, 1)
```

```
In [13]: # NAME : Y.NITHEESH
# REG NO: 212224040370
```

```
class NeuralNet(nn.Module):
    def __init__(self):
        super().__init__()
        # Include your code here
        self.fc1 = nn.Linear(1,8)
        self.fc2 = nn.Linear(8,10)
        self.fc3 = nn.Linear(10,1)
        self.relu = nn.ReLU()
        self.history = {'loss': []}

    def forward(self,x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

```
In [15]: # Initialize the Model, Loss Function, and Optimizer
# Write your code here
lig = NeuralNet()
criterion = nn.MSELoss()
optimizer = optim.Adam(lig.parameters(), lr=0.001)#lr=learning rate
```

```
In [16]: # Name: Y.NITHEESH
# Register Number: 212224040370
def train_model(lig, X_train, y_train, criterion, optimizer, epochs=2000):
    for epoch in range(epochs):
        optimizer.zero_grad()
        loss=criterion(lig(X_train),y_train)
        loss.backward()
        optimizer.step()

        lig.history['loss'].append(loss.item())
    if epoch % 200 == 0:
        print(f'Epoch [{epoch}/{epochs}], Loss: {loss.item():.6f}')
```

```
In [18]: train_model(lig,X_train_tensor, y_train_tensor, criterion, optimizer)
```

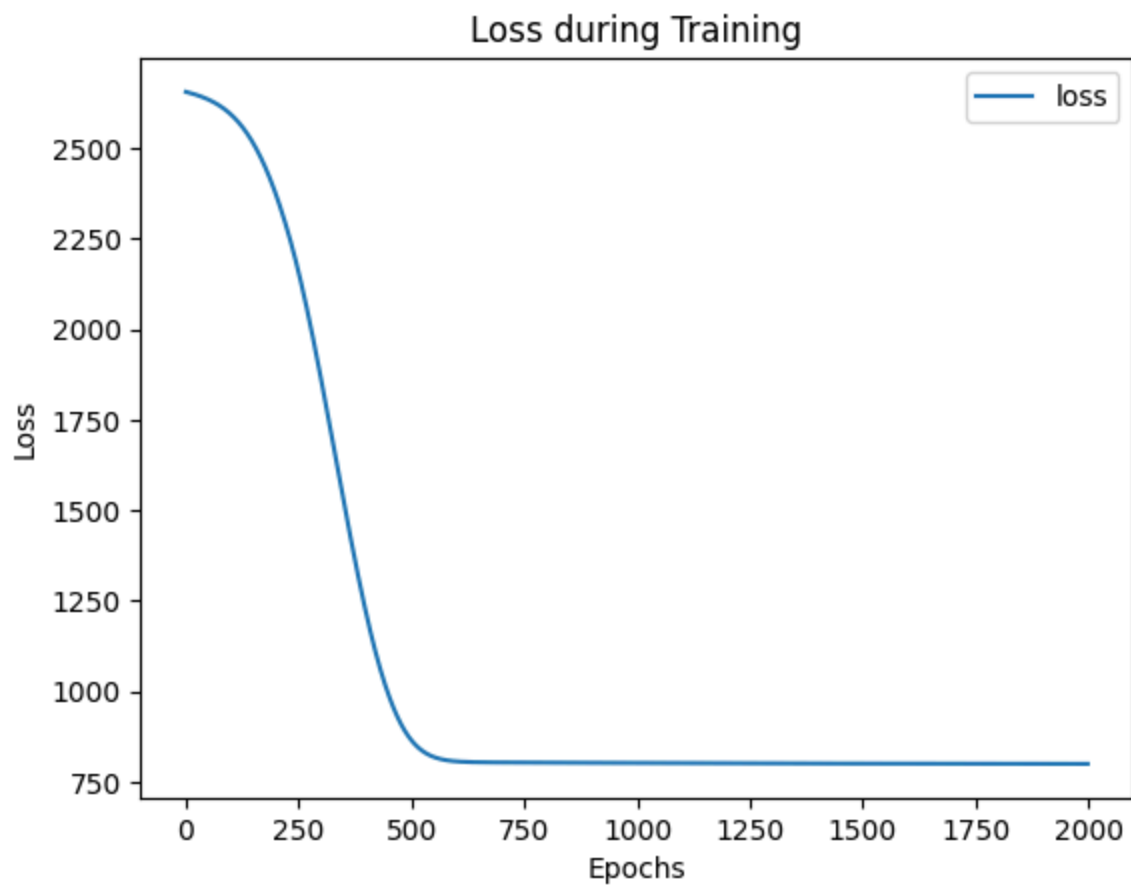
```
Epoch [0/2000], Loss: 2655.373291
Epoch [200/2000], Loss: 2370.476074
Epoch [400/2000], Loss: 1212.861206
Epoch [600/2000], Loss: 805.964294
Epoch [800/2000], Loss: 802.562561
Epoch [1000/2000], Loss: 801.664368
Epoch [1200/2000], Loss: 800.901306
Epoch [1400/2000], Loss: 800.289673
Epoch [1600/2000], Loss: 799.825317
Epoch [1800/2000], Loss: 799.491638
```

```
In [20]: with torch.no_grad():
    test_loss = criterion(lig(X_test_tensor), y_test_tensor)
    print(f'Test Loss: {test_loss.item():.6f}')
```

```
Test Loss: 557.870605
```

```
In [21]: loss_df = pd.DataFrame(lig.history)
```

```
In [22]: import matplotlib.pyplot as plt
loss_df.plot()
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Loss during Training")
plt.show()
```



```
In [24]: X_n1_1 = torch.tensor([[9]], dtype=torch.float32)
         prediction = lig(torch.tensor(scaler.transform(X_n1_1), dtype=torch.float32)).
         print(f'Prediction: {prediction}')
```

Prediction: 29.67989158630371